Clinical Summary Tool

AI-Powered Workflow System · Track 5: Healthcare · README, Submission Guide & Evaluation Rubric

1. Overview

Clinicians spend a lot of time turning long shift notes into handoff summaries. This tool takes a free-text clinical note and returns a **structured, non-diagnostic handoff summary**: a short summary, key points, a **priority level** (Critical / High / Medium / Low), a **risk score** (0–100) with the reasons behind it, and **next steps**. Every result is stored in a database and monitored from an admin dashboard.

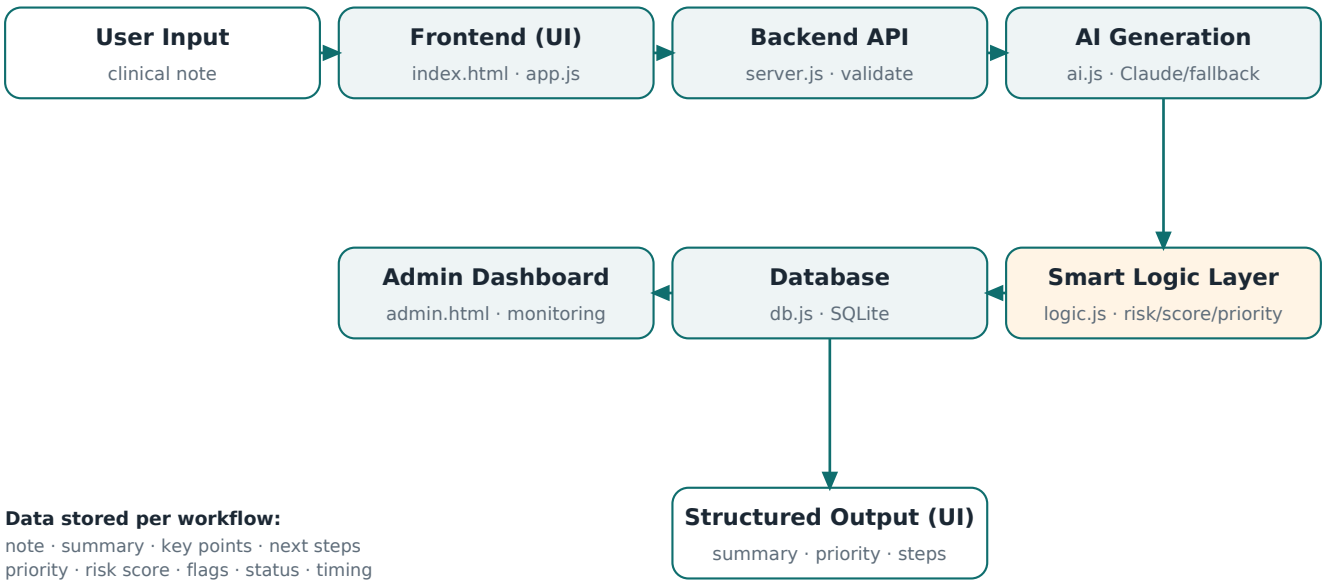
AI is one component of a larger pipeline: Claude (or a built-in rule-based summarizer if no API key is set) writes the summary, while a separate, predictable **Smart Logic Layer** decides how urgent the case is.

Documentation contents

Required topic	Where to find it
System architecture	Section 2: System flow (architecture diagram + stage table)
Frontend implementation	Section 3: Features by layer (Frontend) and Section 6: Screenshots
Backend logic + logic layer	Section 3 (Backend + AI), Section 4: Smart Logic Layer, Section 8: API endpoints
Database design	Section 5: Database design
Admin dashboard overview	Section 3 (Admin dashboard) and Section 6: Screenshots
Git commit progression	Section 10: Git & GitHub
Submission links	Section 11: Submission guidelines

2. System flow

End-to-end architecture. Solid arrows are the main request path; the dashed arrow is the admin monitoring path reading from the same database.



Data stored per workflow:
note · summary · key points · next steps
priority · risk score · flags · status · timing

Stage	What happens	Where in the code
User input	Clinician pastes a note, optional patient ref and author, or picks a sample note	public/index.html
Frontend (UI)	Validates length, sends the note to the API, animates the pipeline strip	public/app.js
Backend API	POST /api/summarize validates input and runs the workflow	server.js
AI generation	Claude returns summary, key points, next steps as JSON; falls back to rule-based if no key or on error	src/ai.js
Smart Logic Layer	Risk keyword detection (with negation handling), vital-sign checks, risk score, priority, action suggestions	src/logic.js
Database	Saves every workflow and an activity log in SQLite	src/db.js → data/clinical.db
Admin dashboard	Stats, priority breakdown, top risks, 7-day usage, activity log, review/delete	public/admin.html, admin.js
Structured output	Priority badge, score bar, risk chips, summary, key points, next steps, copy button	public/index.html

3. Features by layer

Frontend (user screen)

- Note input with character counter, patient reference and author fields
- Four sample notes (critical, high, medium, low) for quick demos
- Live pipeline strip showing Input → API → AI → Logic → Database → Output
- Formatted result card: never raw JSON
- Every next step is tagged **Logic rule**, **AI** or **Summarizer**, so the logic layer’s impact is visible
- “How this result was produced” panel: live trace of each pipeline stage with timing
- Recent summaries table: click any row to reopen it from the database

Backend + AI

- Express API with input validation (20–10,000 characters)
- Claude prompt that forbids diagnosis, invented facts, and medication advice
- Automatic rule-based fallback, so the app works with or without an API key

Smart Logic Layer

- 14 risk rule groups across critical, high and moderate levels
- Negation handling: “denies chest pain” or “no fever” does not raise risk
- Vital-sign parsing: SpO2, heart rate, blood pressure, temperature (°C or °F)
- Explainable score: each point is traced to a category, shown as “Score built from…”
- Safety actions are listed before AI-written steps and de-duplicated

Admin dashboard

- Protected by an admin key
- Totals, today, open, open critical, average score and processing time, errors
- Priority, top risk category and 7-day usage charts
- Activity log including rejected inputs and AI fallbacks

- Search and filter, view full record, mark reviewed / reopen, delete
- Export all workflows to CSV (opens in Excel or Google Sheets)
- Auto-refreshes every 30 seconds

4. Smart Logic Layer: how priority is decided

Level	Examples of keywords / readings	Points
Critical	chest pain, shortness of breath, stroke, seizure, unresponsive, suicidal, sepsis, severe bleeding, SpO2 < 90%, HR > 130 or < 40, systolic BP < 90	30–40
High	fever, hypotension, tachycardia, confusion, fall, low oxygen, allergic reaction, SpO2 90–93%, BP ≥ 180, temp ≥ 39.5°C	15–25
Moderate	pain, nausea, dizziness, headache, swelling, missed dose, poor intake, temp 38–39.4°C	8–10

Priority	Rule	Handoff action added
Critical	Any critical flag, or score ≥ 70	Verbal handoff required
High	Score 40–69	Review this patient first next shift
Medium	Score 15–39	Routine handoff, reassess within shift
Low	Score under 15	Continue current care plan

Why rules and not only AI? Priority must be predictable and explainable: the same note always gets the same priority, and every point of the score has a visible reason.

5. Database design

SQLite: a single-file database (data/clinical.db) created automatically on first run by src/db.js. No setup is needed. It has two tables: one row per processed workflow, and an activity log for monitoring.

Table 1: summaries (one row per workflow)

Column	Type	What it stores
id	INTEGER (primary key)	Auto-numbered workflow ID (#1, #2…)
created_at	TEXT (datetime)	When the note was processed
patient_ref	TEXT	Optional reference such as “Bed 12”
author	TEXT	Optional “Handoff by” name
note_text	TEXT	The original clinical note (input)
summary	TEXT	AI / summarizer summary
key_points	TEXT (JSON list)	Key points as a list
next_steps	TEXT (JSON list)	Next steps, each tagged with its source (logic rule / AI / summarizer)
priority	TEXT	Critical, High, Medium or Low (set by the logic layer)
risk_score	INTEGER	0–100 risk score
risk_flags	TEXT (JSON list)	Each detected risk: category, level, keywords, points
ai_source	TEXT	claude, rule-based, or rule-based (fallback)
processing_ms	INTEGER	Total processing time

Column	Type	What it stores
status	TEXT	open or reviewed (changed from the admin dashboard)

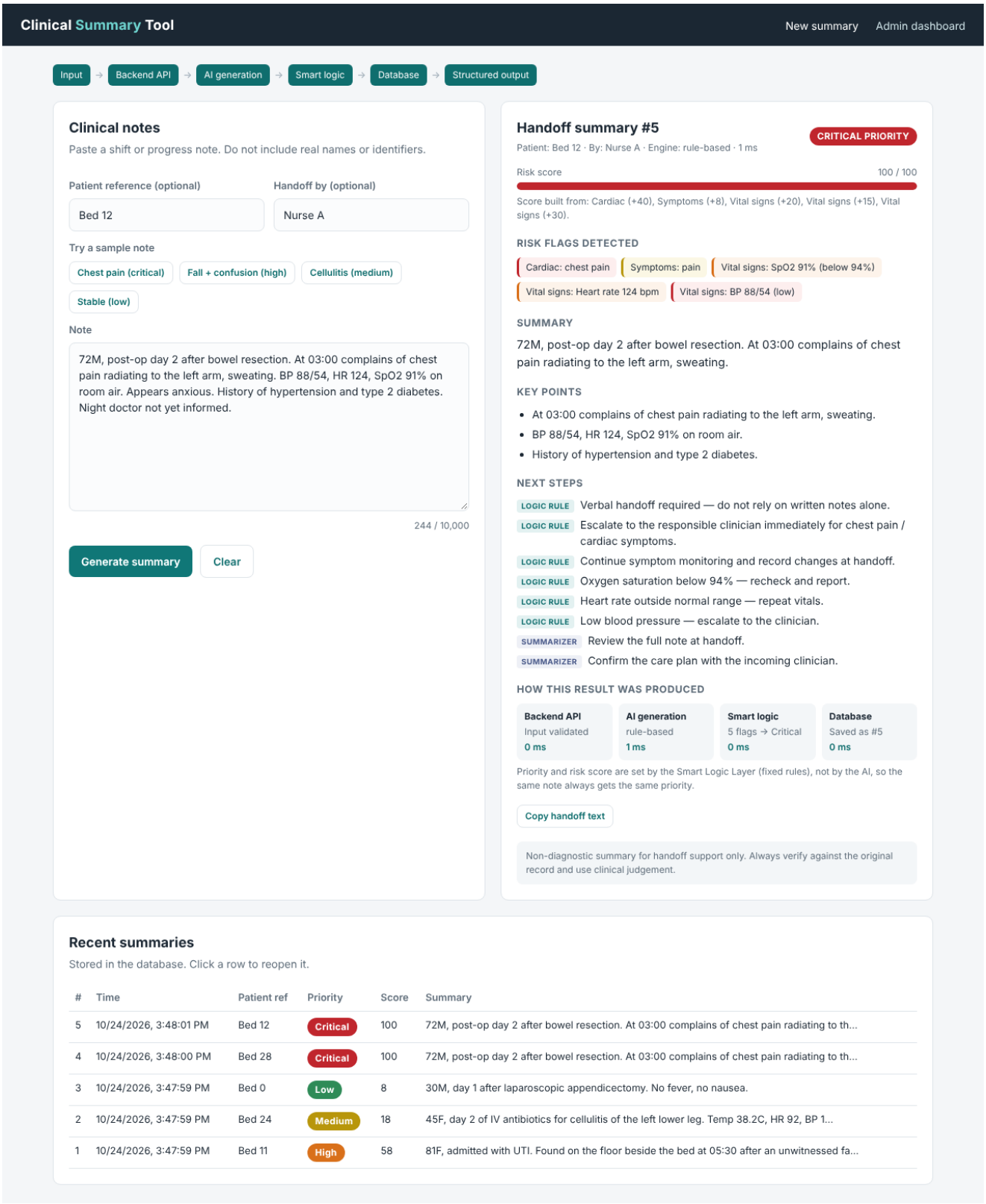
Table 2: activity_log (system monitoring)

Column	Type	What it stores
id	INTEGER (primary key)	Event number
created_at	TEXT (datetime)	When it happened
event	TEXT	e.g. summary_created, summarize_rejected, ai_fallback, status_changed, summary_deleted, export_csv
detail	TEXT	Short description, e.g. “#5 Critical (score 100) via rule-based”
success	INTEGER (1/0)	1 = OK, 0 = error or fallback (counted on the dashboard)

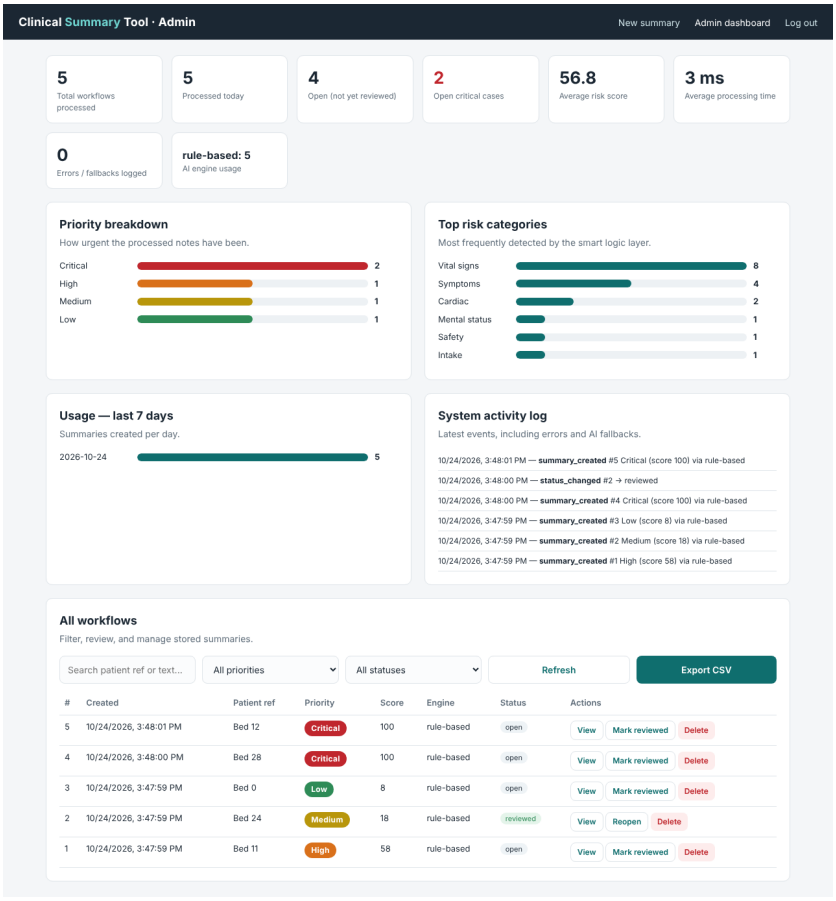
How data moves: **store** on every POST /api/summarize; **retrieve** for Recent summaries, reopening a case, and all admin views; **update** when a case is marked reviewed; **delete** from the admin table; **aggregate** (counts, averages, per-day usage, top risk categories) for the dashboard statistics.

6. Screenshots

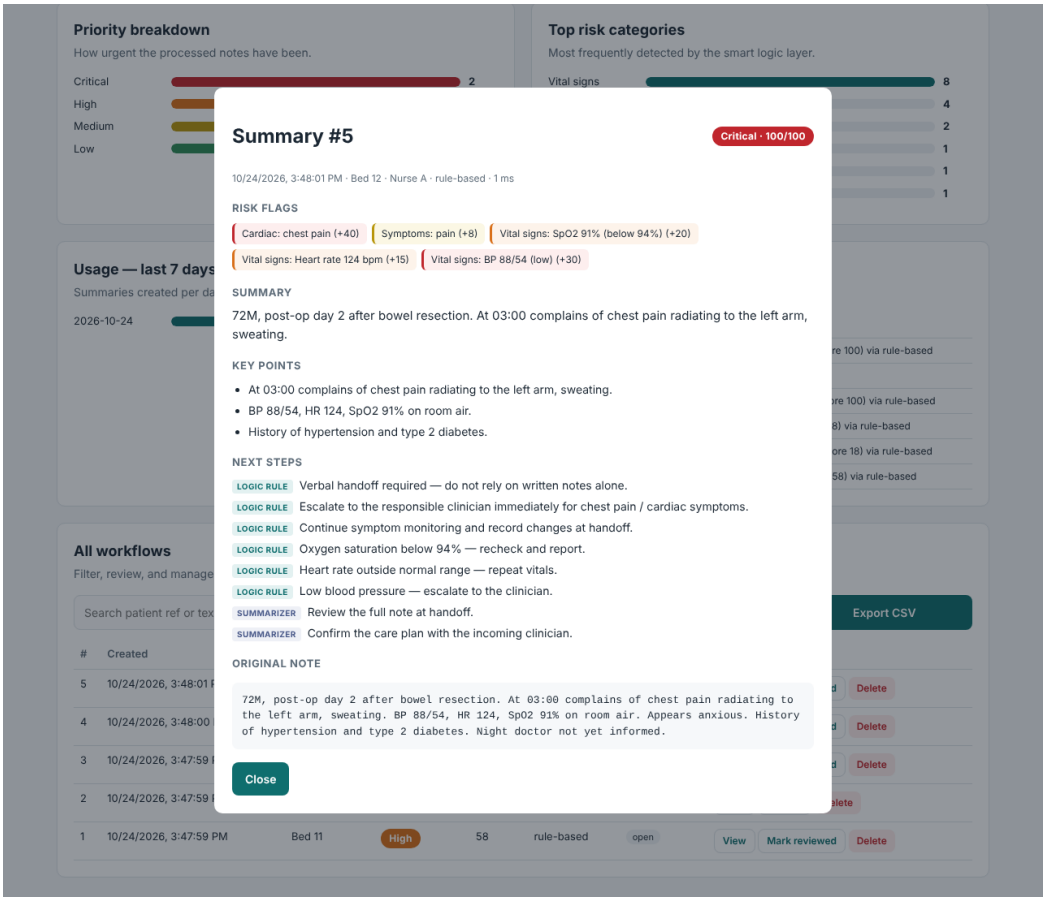
User screen: input on the left, structured output on the right, stored history below.



Admin dashboard: monitoring and managing stored workflows.



Admin detail view: full record with risk flags, tagged next steps and the original note.



7. Tech stack and folder structure

Node.js + Express (backend) · plain HTML/CSS/JavaScript (frontend) · SQLite via better-sqlite3 (database) · Claude API (optional AI) · Replit (development) · Git & GitHub (version control).

```
clinical-summary-tool/
├── server.js           Backend API + workflow orchestration
├── src/ai.js           AI generation (Claude + rule-based fallback)
├── src/logic.js        Smart Logic Layer (risk, score, priority, actions)
├── src/db.js           SQLite database layer
├── public/             index.html, app.js, admin.html, admin.js, styles.css
├── test/run-tests.js  Logic layer tests (npm test)
├── data/               clinical.db is created here automatically
├── .env.example · .replit · .gitignore · package.json
└── README.pdf
```

8. API endpoints

Method	Endpoint	Purpose
GET	/api/health	Server status and AI mode
POST	/api/summarize	Main workflow: note in, structured summary out (saved to DB)
GET	/api/summaries	Recent summaries for the user screen
GET	/api/summaries/:id	One stored summary
GET	/api/admin/stats	Dashboard statistics (needs admin key)
GET	/api/admin/summaries	Filter/search all workflows (needs admin key)
GET	/api/admin/export	Download all workflows as CSV (needs admin key)
PATCH	/api/admin/summaries/:id	Mark reviewed / reopen (needs admin key)
DELETE	/api/admin/summaries/:id	Delete a workflow (needs admin key)

9. Running it on Replit (no coding needed)

1. **Create the Repl.** In Replit, click **Create App** → choose **Import** / upload, and upload the project zip. (Or, once the code is on GitHub, use **Import from GitHub**.)
2. **Add Secrets.** Open the **Secrets** tool (padlock icon) and add **ADMIN_KEY** (your dashboard password). Optionally add **ANTHROPIC_API_KEY** to use Claude. Without it, the rule-based summarizer is used.
3. **Press Run.** Replit installs the packages and starts the server. The web preview opens the user screen.
4. **Open the dashboard** by adding **/admin** to the preview URL and entering your ADMIN_KEY.
5. **Test it.** Click each sample-note button and Generate summary. In the Shell, run **npm test** to run the logic tests (6 should pass).

If Run shows an error about better-sqlite3, open the Shell and type **npm rebuild better-sqlite3**, then press Run again.

10. Git & GitHub: one commit per stage

The project already contains a Git history with one commit per stage (7), followed by improvement commits (4), so the development progression is visible as soon as it is pushed to GitHub.

#	Stage	Commit message	Files added
1	Frontend	Stage 1: Frontend UI for clinical note input and structured output	public/index.html, app.js, styles.css, package.json, config files
2	AI generation	Stage 2: AI generation layer (Claude + rule-based fallback)	src/ai.js
3	Smart logic	Stage 3: Smart logic layer (risk keywords, vitals, scoring, priority)	src/logic.js, test/run-tests.js
4	Database	Stage 4: SQLite database layer for workflows and activity log	src/db.js, data/.gitkeep
5	Admin	Stage 5: Admin dashboard for monitoring and managing workflows	public/admin.html, admin.js
6	Integration	Stage 6: Backend API integrating input → AI → logic → DB → output	server.js
7	Docs	Stage 7: README and submission guide (PDF)	README.pdf
8	Improve	Make logic layer and data flow visible in output: step sources, pipeline trace, CSV export endpoint	server.js, public/app.js, index.html, styles.css
9	Improve	Admin dashboard: CSV export button and step sources in detail view	public/admin.html, admin.js
10	Docs	Docs: add evaluation rubric, architecture diagram and updated screenshots	README.pdf
11	Docs	Docs: add database design section and submission links	README.pdf

Where the history lives

- **GitHub (graded history):** https://github.com/christinemassey3579-ship-it/clinical-summary-tool, with all 11 commits on the main branch. Open it and click **Commits** to see the progression.
- **Replit (running app):** the deployed copy. Replit did not keep the Git history on import, so GitHub is the source of truth for commits. Do not push from Replit to GitHub, as that could overwrite the stage history.

Never commit real API keys. They live only in Replit Secrets; .env is excluded by .gitignore.

11. Submission guidelines

Your submission must include:

- A working full-stack application
- Minimum 6 commits (one per stage)
- Clear frontend ↔ backend ↔ database integration
- Functional admin dashboard
- Structured output (no raw JSON display)
- Visible use of a logic layer

Compliance checklist

Requirement	How this project meets it	How to show it
Working full-stack application	Frontend (public/), Express backend (server.js), SQLite database (src/db.js), all running from one Replit Run	Live Replit URL; generate a summary from a sample note
Minimum 6 commits (one per stage)	11 commits: 7 stage commits (frontend, AI, logic, database, admin, integration, docs) + 4 improvements	GitHub → Commits tab
Frontend ↔ backend ↔ database integration	UI calls POST /api/summarize → AI + logic → saved in DB → returned to UI; history table reads back from the DB	Submit a note, then see it appear in Recent summaries and in the admin table
Functional admin dashboard	/admin with stats, charts, activity log, search, filters, view, mark reviewed, delete	Log in at /admin; mark one case reviewed
Structured output (no raw JSON)	Result card with priority badge, risk bar, risk chips, summary, key points, next steps and a copy button	Screenshot of the result card
Visible use of a logic layer	Risk score, priority, “Score built from…” explanation, risk flag chips and rule-based actions on every result; top risk categories on the dashboard	Compare the four sample notes: each lands on a different priority

What to submit

Item	Link / file
Replit project	https://replit.com/@christinemassey/Clinical-Summary-Tool
Live app	https://clinical-summary-tool.replit.app
Admin dashboard	https://clinical-summary-tool.replit.app/admin
GitHub repository	https://github.com/christinemassey3579-ship-it/clinical-summary-tool
Demo video	Loom or Google Drive link (recorded separately)
Documentation	This README.pdf (also in the GitHub repository)

12. Evaluation rubric

How the project is graded, and where the evidence for each criterion is in this project.

Criterion	Weight	What is assessed
1. System Thinking	30%	End-to-end architecture; logic + data flow
2. Frontend + Backend Integration	20%	Functional interaction; clean UI rendering
3. Database & Admin System	20%	Proper storage and retrieval; admin usability
4. Logic Layer Implementation	15%	Priority/scoring/risk logic; clear impact on output
5. Version Control	10%	Clear commit progression
6. Output Quality & Relevance	5%	Structured and usable output

Evidence for each criterion

Criterion	Evidence in this project	Show the grader
System Thinking (30%)	Separate modules per layer (UI, API, AI, logic, DB, admin) wired in server.js in the exact order of the required flow. Architecture diagram (section 2). AI writes text; rules decide urgency, so each layer has one clear job. Graceful fallback keeps the pipeline working without an API key.	Pipeline strip lights up stage by stage; “How this result was produced” panel shows each stage with timing
Frontend + Backend Integration (20%)	UI posts to /api/summarize and renders the JSON as a formatted card. Input validation on both sides with friendly errors. History table and reopen-by-click read back through the API.	Submit each sample note; click a history row to reopen it
Database & Admin System (20%)	SQLite stores every workflow plus an activity log. Admin: stats, charts, search, filters, detail view, mark reviewed, delete, CSV export, auto-refresh, key-protected.	Mark a case reviewed, filter by Critical, export CSV, view the activity log
Logic Layer Implementation (15%)	14 rule groups, negation handling, vital-sign thresholds, explainable 0–100 score, four priority levels, rule-based actions. Covered by npm test (6 cases).	Next steps tagged “Logic rule”; “Score built from…” line; four samples land on four different priorities
Version Control (10%)	7 stage commits matching the system flow, then 4 clearly named improvement commits.	GitHub → Commits tab
Output Quality & Relevance (5%)	Summary, key points, priority badge, risk bar, risk chips, tagged next steps, copy-ready handoff text, non-diagnostic disclaimer.	Click “Copy handoff text” and paste it

Suggested 3-minute demo (covers every criterion)

1. Show the architecture diagram in this README and name each layer (System Thinking).
2. Load the **Chest pain (critical)** sample and generate. Point out the pipeline strip, the trace panel and the formatted result (Integration, Output).
3. Point to the risk chips, the “Score built from…” line and the **Logic rule** tags on next steps (Logic Layer).
4. Run the **Stable (low)** sample to show the priority change, then reopen an older case from Recent summaries (Logic, Database).
5. Open **/admin**: stats, filters, view a record, mark it reviewed, export CSV (Database & Admin).
6. Open the GitHub Commits tab to show the stage-by-stage progression (Version Control).

13. Safety, privacy and limitations

- Non-diagnostic by design: the tool summarises and flags, it does not diagnose or recommend medication.
- Use synthetic or de-identified notes only. Do not paste real patient names or identifiers.
- Keyword rules can miss unusual wording; output must always be checked against the original record.
- The admin key is a simple password suitable for a demo, not production authentication.